```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;
// Solidity compiler version lock
// Matlab: code isi version ke rules follow karega
// Unexpected behavior future versions me avoid hota hai

pragma abicoder v2;
// ABI Encoder V2 enable
// Isse complex structs safely pass hote hain
// Jaise: MintParams, SwapParams, CollectParams
// Uniswap V3 bina iske kaam hi nahi karta

// ================================
// 🔁 SWAP RELATED
// ================================

import "lib/swap-router-contracts/contracts/interfaces/IV3SwapRouter.sol";
// Uniswap V3 ka swap engine
// Yahin se actual swap hota hai
// exactInputSingle / exactOutputSingle
// FEEL:
// "User ka BUY / SELL order yahin execute hota hai"

// ================================
// 🔐 TOKEN TRANSFER SAFETY
// ================================

import "@uniswap/v3-periphery/contracts/libraries/TransferHelper.sol";
// ERC20 tokens har baar standard nahi hote
// TransferHelper safe wrapper deta hai
// safeTransfer / safeApprove
// Agar token fail kare → transaction revert
// FEEL:
// "Safe courier jo paisa sahi jagah pahunchata hai"

// ================================
// 🏭 POOL CREATION / LOOKUP
// ================================

import "@uniswap/v3-core/contracts/interfaces/IUniswapV3Factory.sol";
```

```
// Factory ka kaam:
// - Check karna pool exist karta hai ya nahi
// - Naya pool banana (token pair + fee tier)
// FEEL:
// "Property registrar jo batata hai plot hai ya naya banana padega"

// ================================
// 📈 LIVE MARKET STATE
// ================================

import "@uniswap/v3-core/contracts/interfaces/IUniswapV3Pool.sol";
// Actual Uniswap V3 pool
// Yahin se real data milta hai:
// - price (sqrtPriceX96)
// - current tick
// - liquidity
// - oracle data (slot0)

// ================================
// 🧾 LP POSITION MANAGER
// ================================

import "@uniswap/v3-periphery/contracts/interfaces/INonfungiblePositionManager.sol";
// LP positions yahin manage hoti hain
// - LP NFT mint
// - liquidity increase / decrease
// - fees collect
// Har LP = ek ERC721 NFT

// ================================
// 🪪 NFT RECEIVE PERMISSION
// ================================

import "@openzeppelin/contracts/token/ERC721/IERC721Receiver.sol";
// Contract ko NFT receive karne ke liye mandatory
// LP position NFT hoti hai
// Agar yeh implement nahi kiya:
// LP mint ke time transaction FAIL ho jaayega

contract SimpleSwap is IERC721Receiver {
```

```
// ========================================================
// 🧩 CONTRACT ROLE
// ========================================================
// Yeh contract IERC721Receiver implement karta hai
// Kyunki Uniswap V3 me LP position = ERC721 NFT hoti hai
//
// FEEL:
// Agar yeh interface implement nahi kiya:
// ❌ LP mint ke time NFT accept nahi hogi
// ❌ Transaction revert ho jaayega
//
// Matlab:
// "Is contract ko NFT receive karne ki permission hai"

// ========================================================
// 📣 EVENTS
// ========================================================

event PoolCreated(
    address indexed tokenA,
    address indexed tokenB,
    uint24 indexed fee,
    address pool
);
// Jab naya Uniswap V3 pool create hota hai
//
// indexed ka matlab:
// - tokenA / tokenB / fee pe frontend easily filter kar sakta hai
//
// Example FEEL:
// ETH / USDC pool create hua
// fee = 3000 (0.3%)

event PoolInitialized(address indexed pool, uint160 sqrtPriceX96);

// Jab pool ka initial price set hota hai (sirf ek baar)
// Yeh event batata hai: "Is pool ki starting price kya thi"
// sqrtPriceX96 = √price × 2^96
// IMPORTANT: Uniswap V3 direct price (2000 USDC) store nahi karta
// Protocol ke andar sirf sqrtPriceX96 store hota hai
```

```
    // Ab REAL VALUES ke saath samjho 👇
    // Maan lo real market me:
    // 1 ETH = 2000 USDC
    // price = 2000
    // Step 1: Square root nikaalo
    // √2000 ≈ 44.72135955
    // Step 2: 2^96 se multiply karo
    // 2^96 ≈ 79,228,162,514,264,337,593,543,950,336
    // Step 3: Final sqrtPriceX96
    // sqrtPriceX96 ≈ 44.72135955 × 2^96
    // ≈ 3543191142285914378072636784640
    // Yeh value integer hoti hai (uint160)
    // Decimal values Solidity me allowed nahi hoti
    // Isliye 2^96 se scale karke precision rakhi jaati hai
    // UI me tum $2000 dekhte ho
    // Contract ke andar yeh badi integer value chal rahi hoti hai
    // Jab pool initialize hota hai:
    // IUniswapV3Pool(pool).initialize(sqrtPriceX96)
    // Us moment pe yeh event emit hota hai
    // Frontend / logs confirm karte hain ki pool ka price set ho gaya
    // Example real scenario:
    // ETH/USDC pool naya bana
    // initialize(3543191142285914378072636784640)
    // Event emit hua:
    // PoolInitialized(poolAddress, 3543191142285914378072636784640)
    // 🧠 ONE-LINE FEEL (LOCK THIS)
    // UI me jo $2000 dikhta hai
    // contract ke andar wahi value sqrtPriceX96 ke form me chal rahi hoti hai

    // ========================================================
    // 🔁 SWAP ROUTER
    // ========================================================

    IV3SwapRouter public immutable swapRouter =
        IV3SwapRouter(0x3bFA4769FB09eefC5a80d6E87c3B9C650f7Ae48E);
    // Uniswap V3 Swap Router ka fixed address
    //
    // Saare swaps:
    // - exactInputSingle
    // - exactOutputSingle
```

```
    // isi router ke through jaate hain
    //
    // immutable = deploy ke baad change nahi ho sakta
    // FEEL: zyada security + kam gas

    // ========================================================
    // 💸 FEE TIER
    // ========================================================

    uint24 public constant feeTier = 3000;
    // Fee tier = 3000 → 0.3%
    //
    // ETH / USDC jaise volatile pairs ke liye
    // yeh most common fee tier hai
    //
    // constant = storage nahi leta → gas efficient

    // ========================================================
    // 🪙 TOKENS
    // ========================================================

    address public immutable token0;
    address public immutable token1;
    // Pool ke tokens
    //
    // Example:
    // token0 = WETH
    // token1 = USDC
    //
    // immutable = constructor me set
    // baad me change nahi ho sakta
    //
    // FEEL:
    // Contract sirf isi token pair ke liye bana hai

    // ========================================================
    // 🏭 FACTORY
    // ========================================================

    IUniswapV3Factory public immutable iUniswapV3Factory =
```

```
        IUniswapV3Factory(0x0227628f3F023bb0B980b67D528571c95c6DaC1c);
    // Uniswap V3 Factory
    //
    // Factory ka kaam:
    // 1️⃣ Check karna → pool exist karta hai ya nahi
    // 2️⃣ Naya pool create karna (token pair + fee)
    //
    // FEEL:
    // "Pool ka registrar office"

    // ========================================================
    // 📃 POSITION MANAGER (LP BOSS)
    // ========================================================

    INonfungiblePositionManager public immutable nonfungiblePositionManager =
        INonfungiblePositionManager(0x1238536071E1c677A632429e3655c799b22cDA52);
    // LP positions ka main controller
    //
    // Yahin se:
    // - LP NFT mint hota hai
    // - liquidity increase / decrease hoti hai
    // - fees collect hoti hain
    //
    // Har LP position = ek ERC721 NFT
    //
    // FEEL:
    // "LP ka bank + demat account"

    // ========================================================
    // 📦 INTERNAL DEPOSIT TRACKING
    // ========================================================

    struct Deposit {
        address owner;
        // LP NFT ka owner kaun hai

        uint128 liquidity;
        // Is LP position ki total liquidity
        // Sirf range ke andar active hoti hai
```

```
        address token0;
        address token1;
    }
    // Is LP position ke tokens
    // Usually contract ke token0 / token1 hi hote hain

    mapping(uint256 => Deposit) public deposits;
    // tokenId (LP NFT id) → Deposit data
    //
    // Example:
    // tokenId = 7
    // owner = Alice
    // liquidity = 123456
    // token0 = WETH
    // token1 = USDC

    // ========================================================
    // 🏗️ CONSTRUCTOR
    // ========================================================

    constructor(address _token0, address _token1) {
        // Contract deploy hote waqt token pair set hota hai
        //
        // Example:
        // _token0 = WETH
        // _token1 = USDC

        require(_token0 != address(0) && _token1 != address(0));
        // Safety check:
        // Zero address ka matlab token exist hi nahi karta
        // Isliye allowed nahi

        token0 = _token0;
        token1 = _token1;
        // Immutable variables set ho gaye
        //
        // FEEL:
        // Ab yeh contract permanently
        // ETH / USDC ke liye locked hai
    }
```

```
function createAndInitializePoolIfNecessary(
    address tokenA,
    address tokenB,
    uint24 fee,
    uint160 sqrtPriceX96
) external returns (address pool) {
    // ========================================================
    // 🧠 STEP 1: Factory se poochte ho
    // ========================================================
    // "Is token pair + fee ka pool pehle se exist karta hai kya?"
    //
    // REAL EXAMPLE:
    // tokenA = WETH
    // tokenB = USDC
    // fee    = 3000 (0.3%)
    //
    // FEEL:
    // Jaise Exchange pe pooch rahe ho:
    // "ETH/USDC market already open hai kya?"
    pool = iUniswapV3Factory.getPool(tokenA, tokenB, fee);

    // ========================================================
    // 🧠 STEP 2: Agar pool nahi mila
    // ========================================================
    // pool == address(0) ka matlab:
    // - Market abhi exist hi nahi karta
    // - No trades possible
    // - No liquidity possible
    if (pool == address(0)) {
        // ========================================================
        // 🧠 STEP 3: Naya pool create karna
        // ========================================================
        // Factory ko bol rahe ho:
        // "ETH/USDC 0.3% ka naya market bana do"
        //
        // IMPORTANT FEEL:
        // Yahan sirf "market ka structure" banta hai
        // Abhi price = 0
        // Trading abhi bhi start nahi hui
```

```
        pool = iUniswapV3Factory.createPool(tokenA, tokenB, fee);

        // ========================================================
        // 🧠 STEP 4: Event emit
        // ========================================================
        // Frontend / logs ko pata chale:
        // "Market create ho gaya"
        emit PoolCreated(tokenA, tokenB, fee, pool);
    }

    // ========================================================
    // 🧠 STEP 5: Pool ka dimaag (slot0) check karna
    // ========================================================
    // slot0 = pool ka brain
    // Isme hota hai:
    // - sqrtPriceX96 (price meter)
    // - current tick
    // - oracle state
    (uint160 currentSqrtPriceX96, , , , , , ) = IUniswapV3Pool(pool)
        .slot0();

    // ========================================================
    // 🧠 STEP 6: Agar price abhi tak set nahi hui
    // ========================================================
    // sqrtPriceX96 == 0 ka matlab:
    // - Pool fresh hai
    // - Trading impossible
    // - LP add nahi ho sakta
    if (currentSqrtPriceX96 == 0) {
        // ========================================================
        // 🧠 STEP 7: Pool initialize karna (PRICE SET)
        // ========================================================
        // sqrtPriceX96 = √price × 2^96
        //
        // REAL MARKET FEEL:
        // 1 ETH = 2000 USDC
        //
        // √2000 ≈ 44.72
        // 2^96 ≈ 7.9e28
        //
```

```
        // sqrtPriceX96 ≈ 3.54e30 (ek badi integer)
        //
        // FEEL:
        // UI me tum $2000 dekhte ho
        // Contract ke andar yahi value "speedometer" ki tarah chalti hai
        IUniswapV3Pool(pool).initialize(sqrtPriceX96);

        // ========================================================
        // 🧠 STEP 8: Event emit
        // ========================================================
        emit PoolInitialized(pool, sqrtPriceX96);
    }

    // ========================================================
    // 🧠 STEP 9: Pool ready
    // ========================================================
    // Ab:
    // - Market exist karta hai
    // - Price set hai
    // - Liquidity add ho sakti hai
    // - Trading start ho sakti hai
    return pool;
}

function mintNewPosition(
    int24 lowerTick,
    int24 upperTick
)
    external
    returns (
        uint256 tokenId,
        uint128 liquidity,
        uint256 amount0,
        uint256 amount1
    )
{
    // ========================================================
    // 🧠 STEP 1️⃣: Pool nikaalna (Market dhoondhna)
    // ========================================================
    // Example:
```

```
        // token0 = WETH
        // token1 = USDC
        // feeTier = 3000 (0.3%)
        //
        // FEEL:
        // "ETH/USDC ka market already bana hai kya?"
        address pool = iUniswapV3Factory.getPool(token0, token1, feeTier);

        // ========================================================
        // 🧠 STEP 2️⃣: Pool ka current price read karna
        // ========================================================
        // slot0 = pool ka dimaag
        // sqrtPriceX96 = √(USDC / ETH) × 2^96
        //
        // REAL MARKET FEEL:
        // ETH ≈ $2000
        // sqrtPriceX96 ≈ 3.54e30 (badi integer)
        (uint160 sqrtPriceX96, , , , , , ) = IUniswapV3Pool(pool).slot0();

        // ========================================================
        // 🧠 STEP 3️⃣: Safety check
        // ========================================================
        // Agar price = 0
        // → pool initialize nahi hua
        // → LP banna allowed nahi
        require(sqrtPriceX96 > 0, "Pool not initialized");

        // ========================================================
        // 🧠 STEP 4️⃣: Tum kitna MAX capital dene ko ready ho
        // ========================================================
        // IMPORTANT:
        // Yeh sirf "upper limit" hai
        // Actual amount price + range pe depend karega
        //
        // Example:
        // "Main max 100 ETH aur 1000 USDC dene ko ready hoon"
        uint256 amount0ToMint = 100 ether; // 100 ETH
        uint256 amount1ToMint = 1000 ether; // 1000 USDC

        // ========================================================
```

```
// 🧠 STEP 5: APPROVE (Permission dena, paisa nahi)
// ==================================================

// User Wallet
//     |
//     | approve
//     ▼
// SimpleSwap Contract
//     |
//     | approve
//     ▼
// NonfungiblePositionManager
//     |
//     | transferFrom
//     ▼
// Uniswap V3 Pool

// ✅ Sahi soch

// "User → Contract → NPM → Pool"

// ETH ke liye approval
// "PositionManager, tum max 100 ETH le sakte ho"
TransferHelper.safeApprove(
    token0,
    address(nonfungiblePositionManager),
    amount0ToMint
);

// USDC ke liye approval
// "PositionManager, tum max 1000 USDC le sakte ho"
TransferHelper.safeApprove(
    token1,
    address(nonfungiblePositionManager),
    amount1ToMint
);

// ❗ IMPORTANT FEEL:
// ❌ Approve = transfer nahi
// ❌ Balance abhi kam nahi hota
```

```
// ✔️ Sirf permission set hoti hai

// ========================================================
// 🧠 STEP 6️⃣: LP STRATEGY define karna (Rule-book)
// ========================================================

// FEEL:
// "Main ETH market me sirf is zone me kaam karunga"

// Example strategy:
// ETH price range = $1800 - $2200

INonfungiblePositionManager.MintParams
    memory params = INonfungiblePositionManager.MintParams({
        token0: token0, // WETH
        token1: token1, // USDC
        fee: feeTier, // 0.3%
        tickLower: lowerTick, // ~ $1800
        tickUpper: upperTick, // ~ $2200
        // MAX capital jo tum ready ho dene ke liye
        amount0Desired: amount0ToMint, // max ETH
        amount1Desired: amount1ToMint, // max USDC
        // amount0Min / amount1Min
        // FEEL:
        // "Isse kam mila to deal cancel"

        // Agar:
        // actualUsed < amountMin
        // → transaction revert
        // → LP NFT mint hi nahi hota

        amount0Min: 0, // demo ke liye ignore
        amount1Min: 0,
        recipient: address(this), // LP NFT yahin aayega
        deadline: block.timestamp
    });

// ========================================================
// 🧠 STEP 7️⃣: ACTUAL MINT (REAL ACTION 🔥)
// ========================================================
```

```
// YAHI MAGIC HOTA HAI:

// Protocol kya dekhta hai?
// - Current price ≈ $2000
// - Tumhari range = $1800-$2200

// Decision:
// - ETH ≈ 2.4
// - USDC ≈ 5200
// - Baaki refund

(tokenId, liquidity, amount0, amount1) = nonfungiblePositionManager
    .mint(params);

// REAL OUTPUT FEEL:
// tokenId  = 42 (LP NFT)
// amount0  ≈ 2.4 ETH
// amount1  ≈ 5200 USDC
// liquidity = ek abstract power number

// ========================================================
// 🧠 STEP 8: Internal accounting (NFT ko yaad rakhna)
// ========================================================
_createDeposit(msg.sender, tokenId);

// ========================================================
// 🧠 STEP 9: Refund + allowance cleanup
// ========================================================
// Agar full ETH use nahi hui
if (amount0 < amount0ToMint) {
    TransferHelper.safeApprove(
        token0,
        address(nonfungiblePositionManager),
        0
    );
    uint256 refund0 = amount0ToMint - amount0;
    TransferHelper.safeTransfer(token0, msg.sender, refund0);
}
```

```
        // Agar full USDC use nahi hui
        if (amount1 < amount1ToMint) {
            TransferHelper.safeApprove(
                token1,
                address(nonfungiblePositionManager),
                0
            );
            uint256 refund1 = amount1ToMint - amount1;
            TransferHelper.safeTransfer(token1, msg.sender, refund1);
        }

        // ========================================================
        // 🧠 FINAL FEEL (LOCK THIS 🔒)
        // ========================================================

        // mintNewPosition =
        // ✔️ LP NFT mint hota hai
        // ✔️ Strategy fixed hoti hai (ticks)
        // ✔️ Capital efficiently deploy hota hai
        // ✔️ Extra funds refund ho jaate hain
    }

    function onERC721Received(
        address operator,
        address,
        uint256 tokenId,
        bytes calldata
    ) external override returns (bytes4) {
        // ========================================================
        // 🧠 YEH FUNCTION KAB CALL HOTA HAI?
        // ========================================================
        // Jab koi ERC721 NFT is contract ko milta hai
        //
        // Uniswap V3 me:
        // LP position = ERC721 NFT
        //
        // REAL SCENARIO:
        // Tumne mintNewPosition() call kiya
        // PositionManager ne LP NFT mint ki
        // NFT ko recipient = address(this) bhej diya
```

```
        //
        // Jaise hi NFT aata hai → yeh function auto-trigger hota hai

        // operator = jisne LP mint ki
        // Example: tumhara wallet
        //
        // tokenId = LP NFT ka unique ID
        // Example: tokenId = 42

        // ========================================================
        // 🧠 STEP 1: LP NFT ka data read karo
        // ========================================================
        _createDeposit(operator, tokenId);

        // FEEL:
        // "NFT mil gayi, ab iska strategy data yaad rakh lo"

        // ========================================================
        // 🧠 STEP 2: Mandatory return
        // ========================================================
        // Yeh return batata hai:
        // "NFT safely receive ho chuki hai"
        //
        // Agar yeh return nahi hua:
        // ❌ transaction revert
        return this.onERC721Received.selector;
    }

    function _createDeposit(address owner, uint256 tokenId) internal {
        // ========================================================
        // 🧠 INPUT FEEL
        // ========================================================
        // owner   = LP banane wala user
        // tokenId = LP NFT ID
        //
        // Example:
        // owner   = Dhruv
        // tokenId = 42

        // ========================================================
```

```
        // 🧠 STEP 1: LP NFT ka locker kholna
        // ========================================================
        (
            ,
            ,
            address posToken0,
            address posToken1,
            ,
            ,
            ,
            uint128 liquidity,
            ,
            ,
            ,

        ) = nonfungiblePositionManager.positions(tokenId);

        // positions(tokenId) ek bada struct return karta hai
        // Usme hota hai:
        //
        // token0     = WETH
        // token1     = USDC
        // tickLower  = 198600 (~$1800)
        // tickUpper  = 199800 (~$2200)
        // liquidity  = 1,234,567,890
        // fees data
        //
        // Commas ( , ) ka matlab:
        // "Is value ko ignore karo"

        // ========================================================
        // 🧠 STEP 2: Apne contract me save karna
        // ========================================================
        deposits[tokenId] = Deposit(owner, liquidity, posToken0, posToken1);

        // ========================================================
        // 🧠 REAL FEEL 🔥
        // ========================================================
        // Ab NFT sirf ek token nahi hai
        // Yeh ek LIVE trading strategy ban chuki hai
```

```
        //
        // Contract ko pata hai:
        // - Kaun owner hai
        // - Kitni liquidity hai
        // - Kaunsa token pair hai
        //
        // FEEL:
        // NFT = on-chain database row
        // Image sirf UI ke liye hoti hai
    }

    function increaseLiquidityCurrentRange(
        uint256 tokenId, // Example: 101 (existing LP NFT)
        uint256 amountAdd0, // Example: 1 ETH
        uint256 amountAdd1 // Example: 2000 USDC
    ) external returns (uint128 liquidity, uint256 amount0, uint256 amount1) {
        // ========================================================
        // 🧠 SCENE SETUP (REAL WORLD FEEL)
        // ========================================================
        // Pool: ETH / USDC (0.3%)
        // Tum already LP ho
        // Tumhare paas LP NFT hai → tokenId = 101
        // Tick range = $1800 - $2200
        // Current ETH price = $2000 (range ke andar ✅)
        //
        // Tum sochte ho:
        // "Is range me volume zyada aa raha hai,
        //  main aur paisa add karta hoon."

        // ========================================================
        // 🔹 STEP 0️⃣ — USER → CONTRACT (ACTUAL TRANSFER)
        // ========================================================
        // FEEL:
        // "User apna paisa pehle mujhe deta hai,
        //  taaki main Uniswap ke saath kaam kar saku"

        // TransferHelper.safeTransferFrom(
        //     token0, // ETH
        //     msg.sender, // User wallet
        //     address(this), // SimpleSwap contract
```

```
//     amountAdd0 // 1 ETH
// );

// TransferHelper.safeTransferFrom(
//     token1, // USDC
//     msg.sender,
//     address(this),
//     amountAdd1 // 2000 USDC
// );

// Ab:
// ✔️ 1 ETH + 2000 USDC contract ke paas hai
// ❌ Abhi Uniswap ko kuch nahi mila

// ========================================================
// 🔹 STEP 1️⃣ — CONTRACT → POSITION MANAGER (APPROVE)
// ========================================================
// FEEL:
// "PositionManager, agar zarurat ho
//  tum mere contract se itna token le sakte ho"

TransferHelper.safeApprove(
    token0,
    address(nonfungiblePositionManager),
    amountAdd0 // max 1 ETH
);

TransferHelper.safeApprove(
    token1,
    address(nonfungiblePositionManager),
    amountAdd1 // max 2000 USDC
);

// ❗ IMPORTANT:
// ❌ approve se token move nahi hota
// ✔️ sirf permission set hoti hai

// ========================================================
// 🔹 STEP 2️⃣ — RULES (SAME STRATEGY)
// ========================================================
```

```
        // Tum bol rahe ho:
        // "Issi LP NFT (tokenId = 101)
        //  ke SAME tick range me
        //  aur liquidity add karo"

        INonfungiblePositionManager.IncreaseLiquidityParams
            memory params = INonfungiblePositionManager
                .IncreaseLiquidityParams({
                    tokenId: tokenId, // 101
                    amount0Desired: amountAdd0, // max ETH
                    amount1Desired: amountAdd1, // max USDC
                    amount0Min: 0, // demo: slippage ignore
                    amount1Min: 0,
                    deadline: block.timestamp
                });

        // ========================================================
        // 🔹 STEP 3️⃣ — ACTUAL EXECUTION (REAL MAGIC 🔥)
        // ========================================================
        (liquidity, amount0, amount1) = nonfungiblePositionManager
            .increaseLiquidity(params);

        // Example REAL RESULT:
        // amount0 = 0.95 ETH
        // amount1 = 1900 USDC
        // liquidity = +350000

        // Matlab:
        // Tumne bola: "max 1 ETH, 2000 USDC"
        // Protocol ne bola: "mujhe sirf itna chahiye"

        // ========================================================
        // 🔹 STEP 4️⃣ — INTERNAL ACCOUNTING
        // ========================================================
        deposits[tokenId].liquidity += liquidity;

        // Example FEEL:
        // Pehle liquidity = 1,000,000
        // Ab liquidity = 1,350,000
        //
```

```
        // ✔️ NFT same
        // ✔️ Range same
        // ✔️ Strategy same
        // 🚀 Fees earning power badh gayi
    }

    function decreaseLiquidityInHalf(
        uint256 tokenId // Example: 101
    ) external returns (uint256 amount0, uint256 amount1) {
        // ========================================================
        // 🧠 SCENE SETUP (REAL FEEL)
        // ========================================================
        // Pool: ETH / USDC (0.3%)
        // tokenId = 101
        // Tick range = $1800 - $2200
        // Current ETH price = $2000 (range ke andar)
        //
        // Tum sochte ho:
        // "Thoda profit nikaal leta hoon,
        //  poora LP band nahi karna"

        // ========================================================
        // 🔹 STEP 1️⃣ — CURRENT LIQUIDITY READ
        // ========================================================
        uint128 liquidity = deposits[tokenId].liquidity;

        // Example:
        // liquidity = 1,350,000

        // ========================================================
        // 🔹 STEP 2️⃣ — HALF CALCULATE
        // ========================================================
        uint128 half = liquidity / 2;

        // Example:
        // half = 675,000
        //
        // FEEL:
        // "Main apni LP ka 50% nikaal raha hoon"
```

```
// ========================================================
// 🔹 STEP 3️⃣ — DECREASE PARAMS
// ========================================================
INonfungiblePositionManager.DecreaseLiquidityParams
    memory params = INonfungiblePositionManager
        .DecreaseLiquidityParams({
            tokenId: tokenId, // 101
            liquidity: half, // 50% liquidity Hold kar do
            amount0Min: 0,
            amount1Min: 0,
            // amount0Min / amount1Min =
            // "Kam se kam itna token mujhe milna hi chahiye,
            // warna liquidity withdraw hi mat karo"

            deadline: block.timestamp
        });

// IMPORTANT:
// ❌ NFT destroy nahi hoti
// ❌ Strategy change nahi hoti
// ✔️ Sirf liquidity kam hoti hai

// ========================================================
// 🔹 STEP 4️⃣ — LIQUIDITY BURN
// ========================================================
nonfungiblePositionManager.decreaseLiquidity(params);

// Pool internally decide karta hai:
// current price = $2000
// kitna ETH + USDC unlock karna hai

// ========================================================
// 🔹 STEP 5️⃣ — COLLECT (ASSETS + FEES)
// ========================================================
(amount0, amount1) = nonfungiblePositionManager.collect(
    INonfungiblePositionManager.CollectParams({
        tokenId: tokenId,
        recipient: address(this), // pehle contract me
        amount0Max: type(uint128).max,
        amount1Max: type(uint128).max
```

```
        })
    );

    // decrease liquidity param me half liquidity ka matlab hai ki 50 percent
liquidity ko hold kardo
    // yani ki pool me lock kar do aur baki bachi huyi 50 percent ko relaease kar
do and amount0Max: type(uint128).max, amount1Max: type(uint128).max
    // ----yaha pe mujhe woh puri 50 de do jo unlocked hai am i iright?----yes

    // Example REAL RESULT:
    // amount0 = 0.48 ETH
    // amount1 = 960 USDC

    //         ❌ Agar MAX na lagate to kya hota?

    // Example:

    // amount0Max = 0.3 ETH
    // amount1Max = 500 USDC

    // Toh:

    // Unlocked tha:
    // 0.48 ETH
    // 960 USDC

    // Tumne bola:
    // "Mujhe max 0.3 ETH, 500 USDC hi do"

    // ➡️ Baaki assets PositionManager ke paas hi pade rehte
    // ➡️ Tum baad me dobara collect() kar sakte ho

    // ========================================================
    // 🔹 STEP 6️⃣ — OWNER KO TRANSFER
    // ========================================================
    _sendToOwner(tokenId, amount0, amount1);

    // FEEL SUMMARY:
    // ✔️ LP NFT same
    // ✔️ Tick range same
```

```
        // ✔️ Strategy same
        // ✔️ 50% capital + fees wallet me
    }

    function collectAllFees(
        uint256 tokenId // Example: 101 (LP NFT)
    ) external returns (uint256 amount0, uint256 amount1) {
        // ========================================================
        // 🧠 SCENE SETUP (REAL WORLD FEEL)
        // ========================================================
        // Pool: ETH / USDC (0.3%)
        // tokenId = 101
        // Tick range = $1800 - $2200
        // Current ETH price = $2000
        //
        // Tum LP ho aur market me trades ho chuke hain
        // Tumhari LP position ne FEES kamayi hain
        //
        // Example fees accumulated:
        // ETH fees  = 0.12 ETH
        // USDC fees = 240 USDC
        //
        // Tum sochte ho:
        // "Liquidity chalu rahe,
        //  bas jo fees bani hain woh nikaal leta hoon."

        // ========================================================
        // 🔹 STEP 0️⃣ — OWNERSHIP REQUIREMENT (IMPLICIT)
        // ========================================================
        // Is function ko call karne wala
        // LP NFT ka owner hona chahiye
        //
        // Kyunki:
        // ❌ koi aur tumhari fees nahi nikaal sakta
        // ✔️ fees sirf owner ki hoti hain

        // ========================================================
        // 🔹 STEP 1️⃣ — NFT TEMPORARILY CONTRACT ME LAANA
        // ========================================================
        // FEEL:
```

```
        // "PositionManager sirf NFT owner se hi
        //  fees collect karne deta hai,
        //  isliye pehle NFT mere paas aani chahiye"

        nonfungiblePositionManager.safeTransferFrom(
            msg.sender, // LP owner (Dhruv)
            address(this), // SimpleSwap contract
            tokenId // 101
        );

        // ❗ IMPORTANT FEEL:
        // ❌ NFT permanently transfer nahi hui
        // ✔️ Sirf function execution ke liye contract ke paas hai
        //
        // Socho jaise:
        // "ATM card thodi der ke liye machine me gaya"

        // ========================================================
        // 🔹 STEP 2️⃣ — COLLECT RULES SET KARNA
        // ========================================================
        // Ab tum bol rahe ho:
        // "Is LP NFT se jitni bhi fees bani hain,
        //  sab nikaal do"

        INonfungiblePositionManager.CollectParams
            memory params = INonfungiblePositionManager.CollectParams({
                tokenId: tokenId, // LP NFT = 101
                recipient: address(this), // Fees pehle contract me aayengi
                amount0Max: type(uint128).max, // ETH fees → ALL
                amount1Max: type(uint128).max // USDC fees → ALL
            });

        // amountMax = MAX ka matlab:
        // "Upper limit mat lagao,
        //  jitni bhi fees hain, sab de do"

        // ========================================================
        // 🔹 STEP 3️⃣ — ACTUAL FEE COLLECTION (MAGIC 🔥)
        // ========================================================
        // Yahin se real transfer hota hai:
```

```
    //
    // Pool → PositionManager → SimpleSwap contract

    (amount0, amount1) = nonfungiblePositionManager.collect(params);

    // Example REAL RESULT:
    // amount0 = 0.12 ETH
    // amount1 = 240 USDC
    //
    // FEEL:
    // "Yeh meri LP ki kamai hai 🎉"

    // ========================================================
    // 🔹 STEP 4️⃣ — FEES OWNER KO BHEJNA
    // ========================================================
    // Contract sirf middleman hai
    // Asli paisa owner ko milna chahiye

    _sendToOwner(tokenId, amount0, amount1);

    // Internally:
    // SimpleSwap → Dhruv wallet
    //
    // ETH  +0.12
    // USDC +240

    // ========================================================
    // 🔹 STEP 5️⃣ — NFT WAPAS OWNER KO
    // ========================================================
    // FEEL:
    // "Kaam ho gaya, NFT wapas le lo"

    nonfungiblePositionManager.safeTransferFrom(
        address(this), // SimpleSwap
        msg.sender, // LP owner
        tokenId // 101
    );

    // ========================================================
    // 🧠 FINAL FEEL SUMMARY (LOCK THIS)
```

```
        // ========================================================
        // ✔️ LP NFT SAME rahi
        // ✔️ Tick range SAME rahi
        // ✔️ Liquidity SAME rahi
        // ✔️ Sirf FEES nikli
        //
        // Yehi Uniswap V3 ka beauty hai:
        // "Earn continuously, withdraw anytime"
    }

    function swapExactInputSingle(
        uint256 amountIn // Example: 1 ETH
    ) external returns (uint256 amountOut) {
        // ========================================================
        // 🧠 SCENE SETUP (REAL WORLD FEEL)
        // ========================================================
        // Pool: ETH / USDC (0.3%)
        // Current market price:
        // 1 ETH ≈ 2000 USDC
        //
        // User bolta hai:
        // "Main EXACT 1 ETH dunga,
        //  jitni USDC mile, mujhe de do"
        //
        // IMPORTANT:
        // ✔️ Input FIXED hai (1 ETH)
        // ❌ Output FIXED nahi hai

        // ========================================================
        // 🔹 STEP 0️⃣ — USER → CONTRACT APPROVAL (ASSUMED)
        // ========================================================
        // User ne pehle hi approve kiya hoga:
        // approve(SimpleSwap, 1 ETH)
        //
        // Agar approve nahi hua → transaction FAIL

        // ========================================================
        // 🔹 STEP 1️⃣ — USER → CONTRACT (ACTUAL TRANSFER)
        // ========================================================
        // FEEL:
```

```
        // "User apna ETH pehle mujhe deta hai,
        //  main Uniswap se swap karwaunga"

        TransferHelper.safeTransferFrom(
            token0, // token0 = ETH (ya WETH)
            msg.sender, // User wallet
            address(this), // SimpleSwap contract
            amountIn // 1 ETH
        );

        // Ab:
        // ✔️ Contract ke paas 1 ETH aa gaya
        // ❌ Abhi Uniswap ko kuch nahi mila

        // ========================================================
        // 🔹 STEP 2️⃣ — CONTRACT → ROUTER (APPROVE)
        // ========================================================
        // FEEL:
        // "SwapRouter, agar zarurat ho,
        //  tum mere contract se max 1 ETH le sakte ho"

        TransferHelper.safeApprove(
            token0,
            address(swapRouter),
            amountIn // 1 ETH
        );

        // ❗ IMPORTANT:
        // approve ≠ transfer
        // Sirf permission set hoti hai

        // ========================================================
        // 🔹 STEP 3️⃣ — SWAP RULES DEFINE KARNA
        // ========================================================
        // Ab tum Uniswap ko EXACT instructions de rahe ho

        IV3SwapRouter.ExactInputSingleParams memory params = IV3SwapRouter
            .ExactInputSingleParams({
                tokenIn: token0, // ETH
                tokenOut: token1, // USDC
```

```
            fee: feeTier, // 0.3% pool
            recipient: msg.sender, // Final USDC user ko milegi
            amountIn: amountIn, // EXACT 1 ETH
            amountOutMinimum: 0, // ❌ Slippage protection nahi
            sqrtPriceLimitX96: 0 // ❌ No price limit
        });

    // 🧠 amountOutMinimum = 0 ka FEEL:
    // "Chahe kitni bhi kam USDC mile,
    //  deal cancel mat karna"

    // 🧠 sqrtPriceLimitX96 = 0 ka FEEL:
    //"Price chahe $1999 ho,
    // chahe $1800 tak gir jaaye,
    // swap complete kar do"

    // Example 1️⃣ — SAFE TRADE

    // Tum bolte ho:

    // "Mujhe 1 ETH swap karna hai,
    // lekin price $1950 se neeche nahi jaana chahiye"

    // Price = 1950 USDC / ETH

    // Convert price → sqrtPriceX96
    // sqrt(1950) ≈ 44.1588
    // sqrtPriceX96 ≈ 44.1588 × 2^96

    // Tum likhte ho:

    // sqrtPriceLimitX96 = 44.1588 * 2^96

    // FEEL:
    // "Jaise hi price $1950 hit kare,
    // swap STOP ho jaaye"

    // ➡️ Agar pool me liquidity kam hui
    // ➡️ Agar sandwich attack hua
    // ➡️ Agar price zyada move karne laga
```

```
    // ❌ Swap revert ho jaayega
    // ✔️ Tumhara ETH safe rahega

    //
    // ⚠️ Production me DANGEROUS
    // (MEV / sandwich attack possible)

    // ========================================================
    // 🔹 STEP 4️⃣ — ACTUAL SWAP (REAL MAGIC 🔥)
    // ========================================================
    // Yahin pe real trade hota hai:
    //
    // Flow:
    // SimpleSwap → SwapRouter → Uniswap Pool
    // Pool se USDC nikal ke user ko milti hai

    amountOut = swapRouter.exactInputSingle(params);

    // ========================================================
    // 🧠 REAL RESULT (EXAMPLE)
    // ========================================================
    // Market price: 2000 USDC / ETH
    // Fee: 0.3%
    // Price impact: thoda sa
    //
    // amountOut ≈ 1985 USDC
    //
    // FEEL:
    // "User ne EXACT 1 ETH diya
    //  aur uske badle ~1985 USDC mili"

    // ========================================================
    // 🧠 FINAL FEEL SUMMARY (LOCK THIS)
    // ========================================================
    // ✔️ Input fixed (1 ETH)
    // ❌ Output variable (depends on price + fee + liquidity)
    // ✔️ LPs earn fees
    // ✔️ Swap instantly execute ho gaya
}
```

```
function swapExactOutputSingle(
    uint256 amountOut, // Example: 2000 USDC (EXACT output)
    uint256 amountInMaximum // Example: 1.05 ETH (max spend limit)
) external returns (uint256 amountIn) {
    // ========================================================
    // 🧠 SCENE SETUP (REAL WORLD FEEL)
    // ========================================================
    // Pool: ETH / USDC (0.3%)
    // Current market price:
    // 1 ETH ≈ 2000 USDC
    //
    // User bolta hai:
    // "Mujhe EXACT 2000 USDC chahiye,
    //  lekin main max 1.05 ETH se zyada nahi dunga"
    //
    // IMPORTANT:
    // ✔️ Output FIXED hai (2000 USDC)
    // ❌ Input FIXED nahi hai (depends on price + fee)

    // ========================================================
    // 🔹 STEP 0️⃣ — USER → CONTRACT APPROVAL (ASSUMED)
    // ========================================================
    // User ne pehle approve kiya hoga:
    // approve(SimpleSwap, 1.05 ETH)
    //
    // Agar approve nahi hua → transaction FAIL

    // ========================================================
    // 🔹 STEP 1️⃣ — USER → CONTRACT (MAX ETH TRANSFER)
    // ========================================================
    // FEEL:
    // "User apna MAX budget pehle contract ko deta hai,
    //  baaki bacha hua paisa baad me refund hoga"

    TransferHelper.safeTransferFrom(
        token0, // token0 = ETH (ya WETH)
        msg.sender, // User wallet
        address(this), // SimpleSwap contract
        amountInMaximum // 1.05 ETH
```

```
    );

    // Ab:
    // ✔️ Contract ke paas 1.05 ETH aa gaya
    // ❌ Abhi nahi pata kitna actually use hoga

    // ========================================================
    //  🔹 STEP 2️⃣ — CONTRACT → ROUTER (APPROVE)
    // ========================================================
    // FEEL:
    // "SwapRouter, tum mere contract se
    //  max 1.05 ETH tak le sakte ho"

    TransferHelper.safeApprove(
        token0,
        address(swapRouter),
        amountInMaximum
    );

    // ========================================================
    //  🔹 STEP 3️⃣ — SWAP RULES DEFINE KARNA
    // ========================================================
    // Ab tum Uniswap ko bol rahe ho:
    // "Mujhe EXACT itni USDC chahiye"

    IV3SwapRouter.ExactOutputSingleParams memory params = IV3SwapRouter
        .ExactOutputSingleParams({
            tokenIn: token0, // ETH
            tokenOut: token1, // USDC
            fee: feeTier, // 0.3%
            recipient: msg.sender, // Final USDC user ko milegi
            amountOut: amountOut, // EXACT 2000 USDC
            amountInMaximum: amountInMaximum, // Max ETH spend = 1.05
            sqrtPriceLimitX96: 0 // ❌ No price limit
        });

    // ========================================================
    //  🔹 STEP 4️⃣ — ACTUAL SWAP (REVERSE MATH 🔥)
    // ========================================================
    // Uniswap kya karta hai:
```

```
        // - Pehle dekhta hai: 2000 USDC dene ke liye
        //   kitna ETH chahiye?
        // - Fee + price impact add karta hai
        // - ETH side se utna hi leta hai

        amountIn = swapRouter.exactOutputSingle(params);

        // ========================================================
        // 🧠 REAL RESULT (EXAMPLE)
        // ========================================================
        // amountOut = 2000 USDC (FIXED)
        //
        // Market condition ke hisaab se:
        // amountIn ≈ 1.01 ETH
        //
        // FEEL:
        // "Mujhe EXACT 2000 USDC mil gayi,
        //  aur sirf 1.01 ETH laga"

        // ========================================================
        // 🔹 STEP 5️⃣ — REFUND EXTRA ETH (VERY IMPORTANT)
        // ========================================================
        // User ne 1.05 ETH diya tha
        // Use hua sirf 1.01 ETH
        // Bacha = 0.04 ETH

        if (amountIn < amountInMaximum) {
            // Router ka allowance reset (good practice)
            TransferHelper.safeApprove(token0, address(swapRouter), 0);

            // Extra ETH user ko wapas
            TransferHelper.safeTransfer(
                token0,
                msg.sender,
                amountInMaximum - amountIn
            );
        }

        // ========================================================
        // 🧠 FINAL FEEL SUMMARY (LOCK THIS)
```

```
        // ==========================================================
        // ✔️ Output FIXED (2000 USDC)
        // ❌ Input VARIABLE (depends on price + fee)
        // ✔️ Extra ETH safely refund ho gaya
        // ✔️ User ka downside risk capped hai
    }

    // ==========================================================
    // 🔢 1⃣ getSqrtPriceX96For1to10Ratio()
    // ==========================================================

    function getSqrtPriceX96For1to10Ratio() public pure returns (uint160) {
        // 🧠 FEEL FIRST:
        // Is function ka kaam hai:
        // "Agar token0 : token1 ka ratio = 1 : 10 ho
        //  to uska sqrtPriceX96 kya hoga?"

        // REAL WORLD EXAMPLE 👇
        // token0 = ETH
        // token1 = USDC
        // 1 ETH = 10 USDC  (sirf demo ratio)

        // Step 1⃣: price calculate
        // price = token1 / token0 = 10 / 1 = 10

        // Step 2⃣: square root
        // sqrt(10) ≈ 3.16227766

        // Step 3⃣: Uniswap scaling
        // sqrtPriceX96 = sqrt(price) × 2^96

        // 2^96 ≈ 79,228,162,514,264,337,593,543,950,336

        // Final value (pre-calculated):
        // 3.16227766 × 2^96
        // ≈ 250541448375047931186413801569

        // 🧠 FEEL:
        // UI me tum "1 ETH = 10 USDC" dekhte ho
        // Contract ke andar yeh badi integer value chalti hai
```

```
    return 250541448375047931186413801569;
}

// ============================================================
// 📍 2⃣ getCurrentTick()
// ============================================================

function getCurrentTick() external view returns (int24) {
    // 🧠 FEEL:
    // Tick = current price ka "floor number"
    // Price move karti hai
    // Tick bas batata hai: "abhi kaunse level pe ho"

    // STEP 1⃣: Pool address nikaal rahe ho
    // Example:
    // token0 = WETH
    // token1 = USDC
    // feeTier = 3000 (0.3%)
    address pool = iUniswapV3Factory.getPool(token0, token1, feeTier);

    // STEP 2⃣: Pool ke brain (slot0) se tick read
    // slot0 ke andar hota hai:
    // - sqrtPriceX96
    // - current tick
    // - oracle data
    (, int24 tick, , , , , ) = IUniswapV3Pool(pool).slot0();

    // REAL FEEL EXAMPLE 👇
    // Maan lo ETH ≈ $2000
    // corresponding tick ≈ 199320
    //
    // Agar ETH $2100 ho jaaye
    // tick ≈ 199740
    //
    // Price smooth hai
    // Tick staircase jaisa

    return tick;
}
```

```
// ============================================================
// 🎯 3️⃣ getTickRangeAroundCurrent()
// ============================================================

function getTickRangeAroundCurrent(
    int24 tickDistance
) external view returns (int24 lowerTick, int24 upperTick) {
    // 🧠 FEEL:
    // "Main current price ke aas-paas
    //  kitna wide LP range rakhun?"

    // STEP 1️⃣: Current tick nikaal lo
    int24 currentTick = this.getCurrentTick();

    // Example:
    // currentTick = 199320 (~$2000)

    // STEP 2️⃣: Tick spacing fix
    // 0.3% fee tier → tickSpacing = 60
    int24 tickSpacing = 60;

    // STEP 3️⃣: Lower tick calculate
    // Example:
    // tickDistance = 300
    // currentTick - 300 = 199020
    // Round DOWN to nearest multiple of 60
    lowerTick = ((currentTick - tickDistance) / tickSpacing) * tickSpacing;

    // STEP 4️⃣: Upper tick calculate
    // currentTick + 300 = 199620
    // Round DOWN to nearest multiple of 60
    upperTick = ((currentTick + tickDistance) / tickSpacing) * tickSpacing;

    // REAL FEEL EXAMPLE 👇
    // currentTick = 199320  (~$2000)
    // tickDistance = 300
    //
    // lowerTick ≈ 199020 (~$1900)
    // upperTick ≈ 199620 (~$2100)
```

```
        //
        // Matlab:
        // "Main $1900 - $2100 ke beech LP banna chahta hoon"

        // 🧠 FEEL:
        // Yeh function tumhe LP strategy auto-calc karke deta hai
    }

    // ============================================================
    // 🏭 4️⃣ getPool()
    // ============================================================

    function getPool(
        address tokenA,
        address tokenB,
        uint24 fee
    ) external view returns (address) {
        // 🧠 FEEL:
        // "Is token pair + fee ka Uniswap V3 pool exist karta hai ya nahi?"

        // REAL EXAMPLE 👇
        // tokenA = WETH
        // tokenB = USDC
        // fee = 3000

        // Agar pool bana hua hai:
        // return = pool address

        // Agar pool nahi bana:
        // return = address(0)

        return iUniswapV3Factory.getPool(tokenA, tokenB, fee);
    }

    // ============================================================
    // 💸 5️⃣ _sendToOwner()
    // ============================================================

    function _sendToOwner(
        uint256 tokenId,
```

```
        uint256 amount0,
        uint256 amount1
    ) internal {
        // 🧠 FEEL:
        // Contract sirf middleman hai
        // Final paisa LP owner ko milna chahiye

        // STEP 1️⃣: LP NFT ka owner nikaalo
        address owner = deposits[tokenId].owner;

        // Example:
        // tokenId = 101
        // owner = Dhruv

        // STEP 2️⃣: token0 transfer
        // Example:
        // amount0 = 0.48 ETH
        TransferHelper.safeTransfer(deposits[tokenId].token0, owner, amount0);

        // STEP 3️⃣: token1 transfer
        // Example:
        // amount1 = 960 USDC
        TransferHelper.safeTransfer(deposits[tokenId].token1, owner, amount1);

        // 🧠 FINAL FEEL:
        // LP owner ke wallet me:
        // - ETH aaya
        // - USDC aayi
        // Contract ka kaam khatam
    }
}
```